Drug review sentiment with manual LSTM cells

Advanced Natural Language Processing · Lab project · 2026/2027

Sara Baalla, Zouhair Eouichak, Malak Bartal, Aymane Youssi
25 October 2026

1. Task objectives

The task predicts negative, neutral or positive sentiment from English patient reviews. We compare a manually implemented Vanilla LSTM baseline with CIFG and WMC variants. Ratings define the expected categories: 1-4 negative, 5-6 neutral and 7-10 positive. These thresholds are a modeling choice and provide proxy sentiment labels. The model receives **review text only**, never the rating, drug name, condition or helpfulness count.

2. Exploratory data analysis

The downloaded Drugs.com files contain **161,297 training reviews** and **53,766 test reviews**, with seven columns. The raw training set has 40,075 negative, 14,356 neutral and 106,866 positive reviews. The test file has 295 missing condition values but no missing review or rating values. Missing conditions do not affect this text-only task.

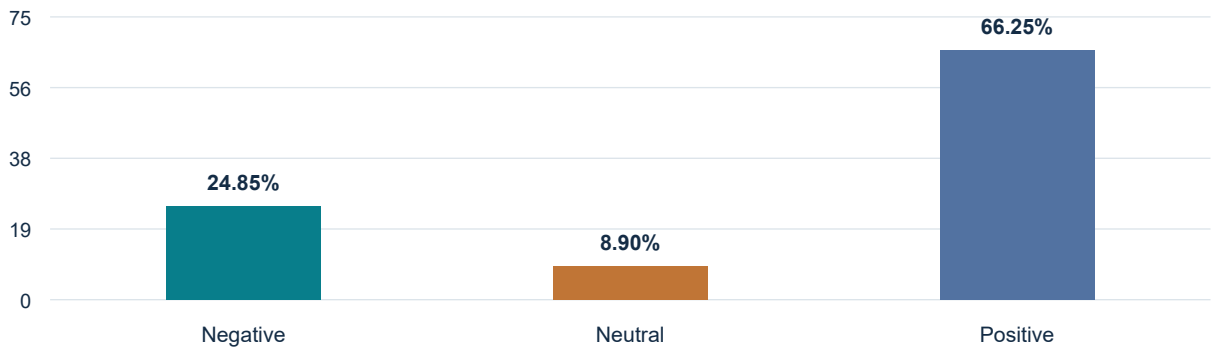


Figure 1. Raw training class proportions. Positive reviews dominate, motivating class-weighted loss and macro-F1 rather than accuracy alone.

HTML encodings such as ' occur in the reviews. Cleaning decodes HTML, lowercases text and normalizes whitespace while retaining punctuation. It excludes 108 training rows whose identical normalized texts have conflicting sentiment labels and removes 48,904 additional duplicates, leaving **112,285 unique texts**. Exact deduplication does not eliminate near-duplicates.

Cleaned pool	Reviews	Experimental subset
Training (85%)	95,443	12,000
Validation (15%)	16,842	2,000
Held-out test	16,153 eligible after cleaning	4,000

The stratified split and subsets use seed 42. All architectures share exactly the same experimental texts. Subsampling preserves category proportions for the CPU training budget. Test cleaning excludes every normalized text present anywhere in the full training or validation pools.

Modeling approach and implementation

3. Text representation and classifier

The vocabulary fits the full 95,443-review training pool only, retaining 11,998 frequent tokens plus PAD=0 and UNK=1. Optimization later uses the fixed 12,000-review subset. Validation and test texts never contribute to the vocabulary. Tokenization preserves internal apostrophes and separates punctuation.

Each input keeps the first **128 tokens**, with padding for shorter reviews. This truncates 33.36% of the full training pool. A trainable Embedding maps IDs to **64-dimensional vectors**. The recurrent hidden state h and memory c each have dimension 64. A linear classifier maps the last useful hidden state to three logits. At padded positions both h and c remain unchanged. No dropout is used in the final study.

Vanilla LSTM: independent gates

For the current embedding x and previous hidden state h_previous, each gate uses its own affine transform. Sigmoid gates lie between 0 and 1. The tanh candidate g proposes new memory content. The same cell weights are reused at every token.

Operation	Equation
Input / forget / output	i, f, o = sigmoid(Wx*x + Wh*h_previous + b), with separate weights
Candidate	g = tanh(Wxg*x + Whg*h_previous + bg)
Memory update	c = f * c_previous + i * g
Hidden output	h = o * tanh(c)

CIFG and WMC: distinct memory mechanisms

CIFG sets i = 1 - f, eliminating the input gate parameters. Retention and insertion are coupled. **WMC** adds tanh-transformed full-matrix memory connections to the input and forget gates from c_previous. Its output gate reads the newly updated c, following the appendix order. The added memory transforms have no extra biases. The symbol * denotes elementwise multiplication in the memory update.

Architecture	Total parameters	Cell parameters
Vanilla	801,219	33,024
CIFG	792,963	24,768
WMC	813,507	45,312

Totals include 768,000 embedding values and 195 classifier parameters. The PAD row remains zero under the optimizer used. Every recurrent affine operation uses **nn.Linear**, without using a built-in LSTM layer to implement the cells. Four tests verify explicit appendix equations, numerical gradients, finite backward gradients, PAD invariance and parameter counts.

Hyperparameter choices and validation

4. Controlled experiment design

All nine runs use Adam, class-weighted cross entropy, gradient clipping at norm 1.0, four epochs and seed 42. The class weight is N/(3*n_class), calculated on training data only. The environment is Python 3.12.14, PyTorch 2.14.1+cpu, CPU and four PyTorch threads. Default PyTorch initialization is fixed by seed.

We compare learning rate and batch size with settings 0.001/128, 0.003/128 and 0.001/64. Relative to the first setting, one factor changes at a time. This does not estimate their full interaction. Architecture selection accounts for differing parameter counts. All other representation and evaluation settings remain shared.

Cell	LR	Batch	Best epoch	Validation F1
VANILLA	0.001	128	4	0.4820
VANILLA	0.003	128	4	0.5050
VANILLA	0.001	64	3	0.4817
CIFG	0.001	128	4	0.4763
CIFG	0.003	128	4	0.5246
CIFG	0.001	64	4	0.4868
WMC	0.001	128	4	0.4934
WMC	0.003	128	4	0.4843
WMC	0.001	64	3	0.4904

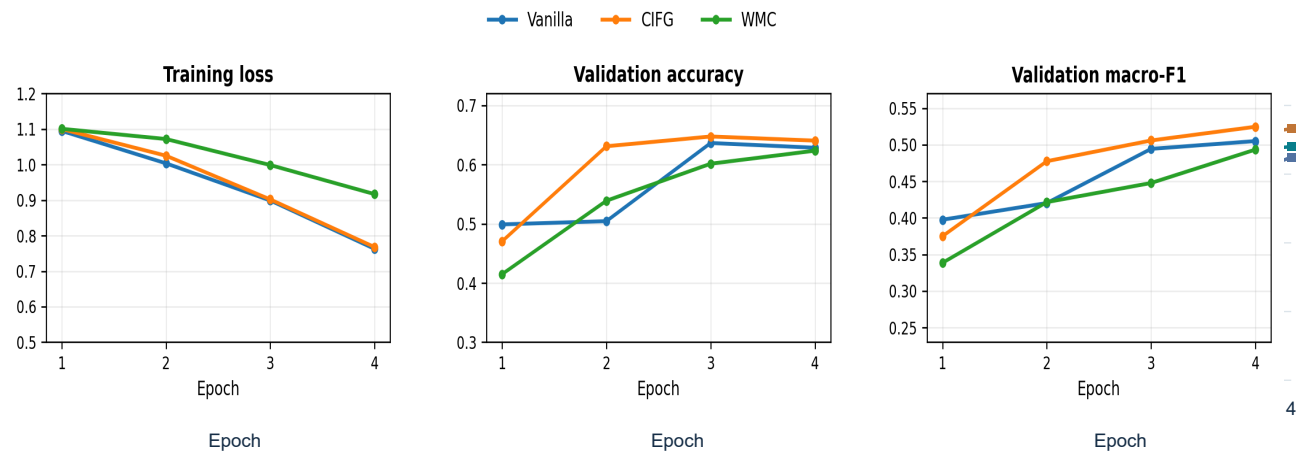


Figure 2. Learning curves for the validation-selected configuration of each architecture: mean training batch loss, validation accuracy and validation macro-F1. All four epochs are observed values. Vertical axes use the displayed restricted ranges. Training loss is class-weighted cross entropy.

Each run retains the epoch with the highest validation macro-F1. Each architecture then selects its best configuration. CIFG leads validation at **0.5246** with LR 0.003 and batch 128. This choice precedes test evaluation. Macro-F1 gives the three categories equal importance, whereas accuracy counts all reviews equally.

Final performance and evaluation

5. Held-out test comparison

Test preparation excludes 15 conflicting rows, collapses duplicates and removes 32,108 normalized unique texts present in the full training/validation pools. From 16,153 eligible texts, a stratified subset of 4,000 contains 988 negative, 356 neutral and 2,656 positive reviews. Test results do not influence configuration selection.

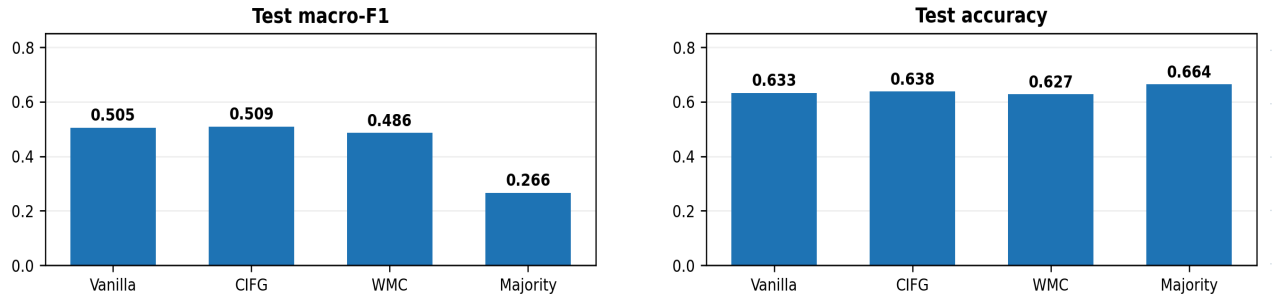


Figure 3. Test macro-F1 and accuracy on the same 4,000 examples, on a 0-1 scale. Majority means always predicting positive. Test evaluation follows validation selection.

Model	Test accuracy	Test macro-F1
VANILLA	63.27%	0.5052
CIFG	63.78%	0.5093
WMC	62.75%	0.4858
Always positive	66.40%	0.2660

		Predicted category		
		Negative	**Neutral**	**Positive**
Actual	**Negative**	**630**	**175**	**183**
	Neutral	**128**	**92**	**136**
	Positive	**366**	**461**	**1829**

Figure 4. Confusion matrix of the validation-selected CIFG model. Rows are actual categories and columns are predictions. Diagonal entries represent correct predictions.

CIFG reaches **63.78% accuracy and 0.5093 macro-F1**. Always predicting positive reaches higher accuracy (66.40%) but much lower macro-F1 (0.2660). The neutral category remains weak: CIFG per-class F1 is 0.5966 for negative, 0.1697 for neutral and 0.7614 for positive. Only 92 of 356 neutral examples are correctly recognized.

The prediction CSVs allow individual error inspection in the same order as test_experience.tsv. Mixed sentiment, rating ambiguity and truncated context are plausible explanations to investigate, not proven causes of every error. The small Vanilla/CIFG gap should not be treated as statistically established with one seed.

Improvements, reproduction and references

6. Conclusions and potential improvements

The project evaluates Vanilla, CIFG and WMC under the same protocol. CIFG achieves the highest validation macro-F1 (0.5246) and the highest observed test macro-F1 (0.5093), versus 0.5052 for Vanilla: a difference of just 0.0041. This is a small descriptive advantage, not evidence of statistically significant superiority from one seed. CIFG also uses fewer trainable parameters (792,963 versus 801,219). WMC adds memory connections but does not improve test macro-F1 under this setup.

The experiment has one random seed, four epochs and a restricted optimization subset. It reports no confidence intervals. Labels derived from ratings can disagree with the review wording, especially for mixed experiences. Neutral reviews are scarce. Exact deduplication does not guarantee independence of near-duplicates or reviews of the same drug, and the 128-token limit discards later context.

Limitation	Next experiment
Single seed / restricted budget	Repeat seeds, report mean and spread, increase data size
Long-review truncation	Compare longer sequences using the same validation protocol
Weak neutral recognition	Inspect ambiguous labels and class-level precision/recall
Representation and generalization	Compare pretrained embeddings and splits grouped by drug

Reproducibility and deliverables

The public repository contains the numbered PyTorch stages, tests, nine experiment histories, final metrics, predictions and ordered subset hashes. The ZIP includes original TSV files, experimental subsets and the three selected checkpoints. The PowerPoint contains 12 slides with English speaker notes. All final scores come from stage 10, not the small verification mode or the earlier educational Vanilla trainer.

Run the download helper, preprocessing stages 3-5, the model tests and stage 10 to reconstruct the study. Source-file SHA-256 hashes and environment versions appear in resultats/synthese.json. For inference, stage 11 reads review text only. The README documents commands, sampling and checkpoint selection.

Public code: https://github.com/baalasara7-max/rnn-drug-review-lstm-2026

References

[1] Course brief: Guidelines_RNN_2627.pdf, Advanced NLP 2026/2027. Modeling requirements and manual cell equations in the appendix.

[2] Kallumadi, S. and Grasser, F. (2018). UCI Drug Review Dataset (Drugs.com), dataset 462. https://doi.org/10.24432/C5SK5S

[3] TSV download mirror: https://github.com/semnan-university-ai/Drug-Review-Dataset. Exact input hashes are recorded. Fresh byte identity with the UCI archive was not independently verified.

[4] CIFG reading supplied in the brief: https://arxiv.org/abs/1503.04069

[5] WMC reading supplied in the brief: https://www.sciencedirect.com/science/article/pii/S0893608021003439

[6] PyTorch Linear and Embedding documentation: https://docs.pytorch.org/docs/stable/nn.html